Календарь: инструкция по API

JSON API, через которое работает веб-интерфейс календаря. Им же можно пользоваться из скриптов и других систем: получать и создавать события, прикреплять файлы, формировать отчёт, искать контакты.

Как начать: войдите под учётной записью домена и сохраните cookie сессии (раздел «Аутентификация» — готовые примеры на `curl` и Python), затем обращайтесь к адресам из сводной таблицы. Работа в интерфейсе описана в USER_GUIDE.md, установка и настройка — в ADMIN_GUIDE.md.

- Базовый адрес — корень сайта, например `http://127.0.0.1:8000`.
- Формат — JSON в кодировке UTF-8 (кроме загрузки и скачивания файлов).
- Все адреса требуют входа в систему и учитывают права пользователя (см. «Права доступа»).

Содержание

1. Аутентификация
2. Общие соглашения
3. Права доступа
4. События
5. Вложения
6. Напоминания
7. Личные настройки
8. Делегирование прав
9. Внешние календари и приглашения
10. Адресная книга (AD / LDAP)
11. Сводная таблица

Аутентификация

API использует сессию Django (cookie `sessionid`), как и интерфейс. Отдельных ключей и токенов API нет: запросы выполняются от имени вошедшего пользователя и с его правами.

Учётная запись — доменная (`ivanov` или `ivanov@domain.ru` и пароль домена); локальный пароль действует только у суперпользователя. Каждый вход проверяется на сервере домена, а неудачные попытки учитываются его политикой блокировки — в скриптах входите один раз и переиспользуйте cookie, а не входите перед каждым запросом. Сессия действует до выхода или две недели.

1. Войти: `POST /accounts/login/` (обычная HTML-форма Django с полями `username`, `password`, `csrfmiddlewaretoken`). Успех — перенаправление `302`; ответ `200` означает, что войти не удалось (страница входа с сообщением об ошибке).
2. Для запросов, которые меняют данные (`POST`, `PUT`, `PATCH`, `DELETE`), передавать CSRF-токен: cookie `csrftoken` + заголовок `X-CSRFToken: <значение cookie>`.
3. Выйти: `POST /accounts/logout/`.

Пример на `curl` (cookie хранятся в файле `jar.txt`):

```
BASE=http://127.0.0.1:8000

# 1) получить cookie csrftoken со страницы входа
curl -s -c jar.txt "$BASE/accounts/login/" > /dev/null
CSRF=$(awk '$6=="csrftoken"{print $7}' jar.txt)

# 2) войти
curl -s -b jar.txt -c jar.txt -X POST "$BASE/accounts/login/" \
     -H "Referer: $BASE/accounts/login/" \
     --data-urlencode "username=ivanov@domain.ru" --data-urlencode "password=секрет" \
     --data-urlencode "csrfmiddlewaretoken=$CSRF" > /dev/null
CSRF=$(awk '$6=="csrftoken"{print $7}' jar.txt)   # токен обновляется после входа

# 3) запросы к API
curl -s -b jar.txt "$BASE/api/events/?start=2026-10-01&end=2026-11-01"

curl -s -b jar.txt -X POST "$BASE/api/events/" \
     -H "Content-Type: application/json" -H "X-CSRFToken: $CSRF" \
     -d '{"title": "Планёрка", "start": "2026-10-12T10:00", "end": "2026-10-12T11:00"}'
```

Пример на Python (`requests`):

```
import requests

BASE = "http://127.0.0.1:8000"
s = requests.Session()
s.get(f"{BASE}/accounts/login/")
s.post(f"{BASE}/accounts/login/",
       data={"username": "ivanov@domain.ru", "password": "секрет", "csrfmiddlewaretoken":
s.cookies["csrftoken"]},
       headers={"Referer": f"{BASE}/accounts/login/"})
assert "sessionid" in s.cookies, "войти не удалось"
s.headers["X-CSRFToken"] = s.cookies["csrftoken"]

events = s.get(f"{BASE}/api/events/", params={"start": "2026-10-01", "end": "2026-11-
01"}).json()["events"]
created = s.post(f"{BASE}/api/events/", json={"title": "Планёрка",
                                              "start": "2026-10-12T10:00", "end": "2026-10-
12T11:00"}).json()
```

Общие соглашения

Дата и время

- Дата и время — строка `YYYY-MM-DDTHH:MM` **в часовом поясе сервера** (`TIME_ZONE`, сейчас `Asia/Yekaterinburg`). Во входных данных допускаются секунды и смещение (`2026-10-12T10:00:00+05:00`).
- Дата без времени (параметры фильтра) — `YYYY-MM-DD`.
- Событие «на весь день» передаётся с временем `00:00` у начала и `23:59` у окончания.

Ошибки

Код	Когда	Тело
`400`	ошибка в данных	`{"errors": {"<поле>": ["сообщение", ...]}}`, общие ошибки — в ключе `__all__`
`401`	не выполнен вход	`{"errors": {"__all__": ["Требуется вход в систему."]}}`
`403`	не хватает прав	`{"errors": {"__all__": ["Нет права ..."]}}`
`403`	нет или неверный CSRF-токен	HTML-страница Django (не JSON)
`404`	объект не найден **или у пользователя нет права его читать**	`{"errors": {"__all__": ["Не найдено."]}}`
`405`	метод не поддерживается	пустое тело, заголовок `Allow` со списком методов

Сообщения об ошибках — на русском, их можно показывать пользователю как есть.

Пример `400`:

```
{"errors": {"title": ["Введите название."], "end": ["Окончание не может быть раньше начала."]}}
```

Права доступа

Подробно — в README, раздел «Роли и права». Для API важно следующее.

Действия:

Действие	Что разрешает в API
Запись	`POST /api/events/` в календарь владельца
Чтение	событие видно в списке и поиске, `GET /api/events/<id>/`, скачивание файлов
Изменение (включает чтение)	`PUT` / `PATCH` / `DELETE` события, загрузка и удаление файлов

Роли пользователя складываются:

- **владелец** — всё в своём календаре;
- **менеджер** — то, что делегировал владелец (на весь календарь или на одно событие);

- **наблюдатель** — чтение и изменение всех событий, без создания;
- **администратор** — всё во всех календарях, может сменить владельца события;
- **приглашённый** на событие всегда может его прочитать.

Каждое событие в ответах содержит итоговые права текущего пользователя:

```
"permissions": {"read": true, "change": false}
```

Событие без права чтения в списке и поиске не возвращается, а при запросе по `id` даёт `404` — так API не раскрывает сам факт его существования.

События

Объект «Событие»

```
{
  "id": 42,
  "title": "Совет директоров",
  "description": "Повестка во вложении",
  "location": "Переговорная 1",
  "url": "https://meet.example.com/abc",
  "guest_emails": ["petr@rambler.ru", "sidorova@domain.ru"],
  "directory_guests": ["sidorova@domain.ru"],
  "participant_names": {"sidorova@domain.ru": "Сидорова Мария"},
  "reminder_minutes": 15,
  "start": "2026-10-12T10:00",
  "end": "2026-10-12T11:30",
  "all_day": false,
  "color": "blue",
  "owner": {"id": 3, "name": "Иван Петров"},
  "invitees": [{"id": 5, "name": "Анна Петрова", "email": "apetrova@domain.ru"}],
  "attachments": [
    {"id": 7, "name": "повестка.pdf", "size": 183204, "url": "/api/attachments/7/download/"}
  ],
  "permissions": {"read": true, "change": true}
}
```

Поле	Тип	Запись	Описание
`id`	число	—	идентификатор
`title`	строка ⩽ 200	**обязательно**	название, не может состоять из одних пробелов
`description`	строка	необязательно	описание
`location`	строка ⩽ 200	необязательно	место

Поле	Тип	Запись	Описание
`url`	строка ⩽ 500	необязательно	ссылка; без схемы дополняется `https://` (`zoom.us/j/1` → `https://zoom.us/j/1`)
`guest_emails`	строка или список	необязательно	внешние участники: адреса через запятую, точку с запятой, пробел или перевод строки (до 100, повторы убираются); в ответе — список. Им можно отправить приглашение, см. «Приглашения»
`directory_guests`	список e-mail	необязательно	кто из гостей и приглашённых выбран из адресной книги AD/LDAP: им событие автоматически ставится в календарь (собранием Exchange или письмом с `.ics`) и обновляется при изменениях. Учитываются только адреса из `guest_emails` и адреса приглашённых сотрудников; в ответе — в нижнем регистре. Если поле не передано, пометка сохраняется прежней. Сотрудник с таким e-mail получает право чтения события
`participant_names`	объект `{e-mail: имя}`	необязательно	имена участников для показа (из адресной книги); сохраняются только для адресов из `guest_emails`
`reminder_minutes`	число или `null`	необязательно	напоминание за N минут до начала: `0` , `5` , `10` , `15` , `30` , `60` , `120` , `1440` , `2880` , `10080` ; `null` — без напоминания. Если поле не передано, прежнее значение сохраняется. Передаётся в Exchange (Outlook напомнит сам) и в приглашения `.ics`
`start`	дата-время	**обязательно**	начало
`end`	дата-время	необязательно	окончание, не раньше `start` ; если не задано — в ответе равно `start`
`all_day`	логическое	необязательно, `false`	событие на весь день
`color`	строка	необязательно, `blue`	один из 24 цветов: `red` , `crimson` , `wine` , `pink` , `fuchsia` , `purple` , `indigo` , `navy` , `blue` , `sky` , `cyan` , `teal` , `emerald` , `green` , `lime` , `olive` , `yellow` , `orange` , `tangerine` , `brown` , `sand` , `gray` , `slate` , `graphite` . Во внешние календари и участникам не передаётся
`invitees`	список `id` пользователей	необязательно	приглашённые; в ответе — список `{id, name, email}` . **Если поле не передано**, приглашёнными становятся пользователи календаря, чьи e-mail есть в `guest_emails` (так работает поле «Участники» в форме); владелец события приглашённым не бывает

Поле	Тип	Запись	Описание
`owner`	`id` пользователя	только при создании (и для администратора — при изменении)	чей это календарь; в ответе — `{id, name}` или `null`
`attachments`	—	только чтение	файлы, см. «Вложения»
`permissions`	—	только чтение	права текущего пользователя
`delivery`	—	только в ответе на создание/изменение	итог доставки участникам из `directory_guests`: `{"channel": "exchange"|"email", "invited": [...], "updated": [...], "cancelled": [...], "failed": {"адрес": "причина"}, "errors": [...], "notes": [...]}`. Гость, вписанный вручную, но найденный в адресной книге, отмечается в `directory_guests` автоматически
`restricted`	—	только чтение	`true`, если у пользователя нет права чтения: подробности (описание, место, ссылка, участники, файлы) пустые. Встречается, например, в ответе на создание у менеджера с одной «Записью»

Список событий и поиск

```
GET /api/events/?start=YYYY-MM-DD&end=YYYY-MM-DD&q=текст
```

Параметр	Описание
`start`	события, которые заканчиваются не раньше начала этого дня
`end`	события, которые начинаются раньше начала этого дня (граница не включается)
`q`	поиск по названию, месту и описанию; без учёта регистра, в том числе для кириллицы

Все параметры необязательны и сочетаются. Возвращаются только события, которые пользователь может читать, отсортированные по началу, не больше 2000.

Ответ `200`:

```
{"events": [ {...событие...}, ... ]}
```

Примеры:

```
GET /api/events/?start=2026-10-01&end=2026-11-01     — всё за октябрь 2026
GET /api/events/?start=2026-10-12&end=2026-10-13     — события 12 октября, включая
многодневные
GET /api/events/?q=планёрка                          — поиск по всему календарю
```

Отчет по событиям (.docx)

```
GET /api/events/report/?start=2026-10-01&end=2026-10-31
```

Файл Word «Отчет по событиям.docx» (`Content-Disposition: attachment`): все события, которые пользователь может читать — то же, что он видит в календаре, — за период с `start` по `end` **включительно**, кроме событий календарей, выключенных в фильтре календарей (настройка `hidden_calendars` , см. «Личные настройки»); событие попадает в отчет, если пересекается с периодом. Оба параметра обязательны. В документе — период, пользователь, число событий и таблица по времени начала: дата, время, событие (тема, описание, ссылка), место, участники, календарь (владелец). В интерфейсе — кнопка «Отчет» в левой панели «Фильтр календарей» и в окне «Настройки».

Ошибки: `400` — период не указан, дата некорректна или окончание раньше начала.

Получить событие

```
GET /api/events/<id>/
```

Нужно право **Чтение**. Ответ `200` — объект события; `404` , если события нет или его нельзя читать.

Создать событие

```
POST /api/events/
Content-Type: application/json
```

Нужно право **Запись** в календарь владельца. По умолчанию событие создаётся в календаре текущего пользователя; чтобы создать в чужом (менеджер, администратор), укажите `owner` .

```
{
  "title": "Совет директоров",
  "start": "2026-10-12T10:00",
  "end": "2026-10-12T11:30",
  "location": "Переговорная 1",
  "url": "https://meet.example.com/abc",
  "color": "green",
  "invitees": [5, 8],
  "owner": 3
}
```

Ответ `201` — созданное событие. Ошибки: `400` (данные), `403` (нет права записи в этот календарь).

Изменить событие целиком

```
PUT /api/events/<id>/
Content-Type: application/json
```

Нужно право **Изменение**. Передаётся событие целиком, как при создании: необязательные поля, которых нет в запросе, получают значения по умолчанию — текстовые очищаются, `end` сбрасывается, `all_day` становится `false`, `color` — `blue`, а отсутствие `invitees` удаляет всех приглашённых. Чтобы поменять только часть полей, используйте `PATCH`.

`owner` можно передать, но сменить владельца может только администратор (иначе `403`).

Ответ `200` — событие после изменения.

Изменить часть полей

```
PATCH /api/events/<id>/
Content-Type: application/json
```

Нужно право **Изменение**. Меняются только переданные поля, остальные сохраняются. Так интерфейс переносит событие перетаскиванием:

```
{"start": "2026-10-14T10:00", "end": "2026-10-14T11:30"}
```

Ответ `200` — событие после изменения.

Удалить событие

```
DELETE /api/events/<id>/
```

Нужно право **Изменение**. Удаляются и все вложения события (вместе с файлами). Ответ `200`:

```
{"deleted": 42}
```

> **Внешние календари.** После создания, изменения (`POST`, `PUT`, `PATCH`) и удаления события оно сразу, в фоне, отправляется во все подключённые календари владельца с направлением «в обе стороны» или «от нас туда» (например, в Microsoft Exchange). Ответ API этого не ждёт; ошибки отправки видны в поле `last_error` подключения, а недоставленное досылает плановая синхронизация. Так же, в фоне, событие доставляется в календари участников из `directory_guests`: новым — приглашение, при изменении содержимого — обновление всем, удалённым из списка и при удалении события — отмена.

Вложения

Файлы событий. Хранятся в `MEDIA_ROOT/attachments/ГГГГ/ММ/`, отдаются только через API.

Объект «Вложение» (в событии, в поле `attachments`):

```
{"id": 7, "name": "повестка.pdf", "size": 183204, "url": "/api/attachments/7/download/"}
```

`size` — в байтах, `url` — адрес для скачивания.

Загрузить файлы

```
POST /api/events/<id>/attachments/
Content-Type: multipart/form-data
```

Поле формы `files`, можно несколько файлов за раз. Ограничения: до **20 МБ** на файл, до **20 файлов** на событие.

Нужно право **Изменение**. Исключение: автор только что созданного события, у которого есть право **Запись** в этот календарь, может прикрепить к нему файлы и без права изменения.

```
curl -s -b jar.txt -X POST "$BASE/api/events/42/attachments/" -H "X-CSRFToken: $CSRF" \
     -F "files=@повестка.pdf" -F "files=@смета.xlsx"
```

Ответ `201`:

```
{"attachments": [ {...вложение...}, {...вложение...} ]}
```

Ошибки `400` — в поле `files`: файлы не выбраны, превышен размер или количество.

Файлы и приглашения. Файлы события вкладываются в письмо-приглашение участникам из адресной книги. После загрузки или удаления файла участникам уходит приглашение с файлами (если оно ещё не отправлялось) или обновление; итог — в поле `delivery` ответа (как при сохранении события). Чтобы новое событие с файлами пришло участникам одним письмом, при сохранении события передайте `"files_pending": true` — приглашение будет отправлено после операции с файлами. Если операций с файлами несколько, добавьте `?deliver=0` ко всем, кроме последней.

Скачать файл

```
GET /api/attachments/<id>/download/
```

Нужно право **Чтение** события. Ответ — содержимое файла с заголовком `Content-Disposition: attachment` и исходным именем (файл всегда скачивается, а не открывается в браузере).

Удалить файл

```
DELETE /api/attachments/<id>/
```

Нужно право **Изменение** события. Файл удаляется и с диска. Ответ `200`: `{"deleted": 7}`.

Напоминания

```
GET /api/reminders/
```

Напоминания текущего пользователя, которые пора показать или покажутся в ближайшие 15 минут — по событиям, где он владелец или участник и задано `reminder_minutes`. Страница календаря опрашивает раз в минуту и показывает уведомление.

```
{"reminders": [{"id": 42, "title": "Тест", "start": "2026-10-12T10:00", "all_day": false,
"location": "каб. 500",
                "minutes": 15, "remind_at": "2026-10-12T04:45:00+00:00"}], "now": "..."}
```

Личные настройки

У каждого пользователя свои; хранятся на сервере.

```
GET   /api/me/settings/
PATCH /api/me/settings/   {"theme": "dark"}
```

Ответ: `{"settings": {"theme": "system", "start_view": "last", "last_view": "month", "hidden_calendars": []}}`

Поле	Значения	Описание
`theme`	`system`, `light`, `dark`	оформление
`start_view`	`last`, `month`, `week`, `day`	вид при открытии календаря; `last` — последний выбранный
`last_view`	`month`, `week`, `day`	последний выбранный вид (страница сохраняет его сама при переключении)
`hidden_calendars`	список id пользователей, например `[12, 40]`; по умолчанию `[]`	фильтр календарей: календари этих владельцев выключены. Страница не показывает их события, а `/api/events/report/` не включает их в отчет. Повторы убираются, список сортируется; не больше 1000 id. Передаётся целиком — заменяет прежний список

Неизвестное поле или значение (в том числе `hidden_calendars` не списком положительных целых чисел) — `400`, ничего не сохраняется.

Пример — выключить календарь пользователя с id 40 и снова включить все:

```
PATCH /api/me/settings/   {"hidden_calendars": [40]}
PATCH /api/me/settings/   {"hidden_calendars": []}
```

`GET /api/events/` фильтр календарей не применяет — возвращает все доступные события; скрывает их сама страница. Пункты фильтра (свой календарь и календари владельцев, давших право «Чтение»; администратору, наблюдателю и менеджеру всех календарей — все календари с событиями) страница получает при загрузке, отдельного API для них нет.

Делегирование прав

Права, которые владелец выдаёт другим сотрудникам **на весь свой календарь** (сотрудник становится менеджером этого календаря). Права на отдельное событие настраиваются в админке (`/admin/` , раздел «Делегирование прав»).

Все адреса этого раздела доступны владельцу календаря или администратору; остальным — `403` .

Объект «Делегирование»

```
{
  "id": 12,
  "manager": 5,
  "user": {
    "id": 5, "name": "Анна Петрова", "first_name": "Анна", "last_name": "Петрова",
    "email": "anna@corp.ru", "username": "apetrova"
  },
  "event": null,
  "can_create": true,
  "can_read": true,
  "can_change": false
}
```

Поле	Описание
`manager`	`id` сотрудника, которому выданы права
`user`	данные сотрудника
`event`	`null` — права на весь календарь
`can_create`	**Запись** — создавать события в календаре владельца
`can_read`	**Чтение**
`can_change`	**Изменение**; если `true` , `can_read` тоже становится `true`

Кому выданы права

```
GET /api/delegations/
```

Ответ `200` — права, выданные текущим пользователем на весь его календарь, по фамилии сотрудника:

```
{"delegations": [ {...делегирование...}, ... ]}
```

Выдать, изменить или отозвать права

```
POST /api/delegations/
Content-Type: application/json
```

```
{"manager": 5, "can_create": true, "can_read": true, "can_change": false}
```

- Если права этому сотруднику ещё не выданы — создаются, иначе — заменяются переданными.
- Все флаги `false` — права отзываются.
- Флаги, которых нет в запросе, считаются `false`.

Ответ `200` — делегирование после сохранения (при отзыве — `{"manager": 5, "can_create": false, "can_read": false, "can_change": false}`). Ошибки `400`: `manager` не указан, сотрудник не найден, неактивен или это сам владелец.

Поиск сотрудника (подсказки)

```
GET /api/delegations/candidates/?q=текст&limit=10
```

Поиск по фамилии, имени (в любом порядке), e-mail или логину — без учёта регистра, «ё» = «е». Текущий пользователь и неактивные сотрудники не возвращаются. `limit` — от 1 до 50, по умолчанию 10.

Порядок результатов — от лучшего совпадения к худшему:

1. полное совпадение фамилии, имени, «имя фамилия», «фамилия имя», e-mail или логина (`"exact": true`);
2. начало фамилии, имени, e-mail или логина;
3. начало «фамилия имя» / «имя фамилия»;
4. каждое слово запроса — начало какого-то слова («иван пет»);
5. запрос встречается где угодно.

Внутри одной группы — по фамилии и имени.

Ответ `200`:

```
{
  "query": "петров",
  "results": [
    {"id": 3, "name": "Иван Петров", "first_name": "Иван", "last_name": "Петров",
     "email": "ivan.petrov@corp.ru", "username": "ipetrov", "exact": true},
    {"id": 5, "name": "Анна Петрова", "first_name": "Анна", "last_name": "Петрова",
     "email": "anna@corp.ru", "username": "apetrova", "exact": false}
  ]
}
```

Пустой `q` — пустой список.

Внешние календари и приглашения

Подробно о том, как устроена синхронизация, — в INTEGRATIONS.md. Все адреса работают с внешними календарями **текущего пользователя**; подключать их может владелец своего календаря (или администратор), остальным — `403`. Чужой подключённый календарь даёт `404`.

Объект «Внешний календарь»

```
{
  "id": 3, "provider": "yandex", "provider_label": "Яндекс Календарь",
  "calendar_name": "Работа", "server_url": "https://caldav.yandex.ru", "username":
"ivan@yandex.ru",
  "direction": "both", "direction_label": "В обе стороны", "color": "gray",
  "remote_read_only": false, "enabled": true,
  "last_sync_at": "2026-10-10T23:40:12.123456+05:00", "last_error": "",
  "last_summary": "к нам: +3 ~0 −0; туда: +1 ~0 −0"
}
```

`provider`: `google`, `exchange` (Microsoft Exchange, EWS), `yandex`, `mailru`, `rambler`, `caldav`. `direction`: `both`, `import` (только к нам), `export` (только от нас). Пароль и токены в ответах не возвращаются.

Список и доступные сервисы

```
GET /api/integrations/
```

```
{
  "calendars": [ {...внешний календарь...} ],
  "providers": [{"code": "yandex", "label": "Яндекс Календарь", "auth": "password",
                 "server_url": "https://caldav.yandex.ru", "available": true, "help": "..."}],
  "google_pending": false,
  "colors": [{"code": "blue", "label": "Синий"}],
  "directions": [{"code": "both", "label": "В обе стороны"}]
}
```

`available: false` у Google — не заданы ключи OAuth. `google_pending: true` — пользователь вошёл через Google, но ещё не выбрал календарь.

Найти календари (проверка входа)

```
POST /api/integrations/discover/
```

Exchange: `{"provider": "exchange", "server_url": "https://email.domain.ru/EWS/Exchange.asmx", "username": "YG\\ivanov", "password": "..."}` — `id` основного календаря `calendar`.

CalDAV: `{"provider": "yandex", "server_url": "https://caldav.yandex.ru", "username": "ivan@yandex.ru", "password": "пароль приложения"}` (`server_url` можно не передавать — подставится адрес по умолчанию). Google: `{"provider": "google"}` после входа через Google.

Ответ `200`:

```
{"calendars": [{"id": "https://caldav.yandex.ru/calendars/ivan%40yandex.ru/events-123/",
"name": "Мои события",
                "color": "#FF2968", "read_only": false, "connected": false}]}
```

Ошибка `400` с понятным текстом: неверный пароль, сервер недоступен, календари не найдены.

Подключить календарь

```
POST /api/integrations/
```

Те же поля, что для `discover`, плюс выбранный календарь:

```
{"provider": "yandex", "username": "ivan@yandex.ru", "password": "...",
 "calendar_id": "<id из discover>", "calendar_name": "Мои события",
 "direction": "both", "color": "green", "read_only": false}
```

Сразу выполняется первая синхронизация. Ответ `201`: `{"calendar": {...}, "sync": {...итог...}}`. Повторное подключение того же календаря — `400`.

Итог синхронизации:

```
{"created_local": 3, "updated_local": 0, "deleted_local": 0,
 "created_remote": 1, "updated_remote": 0, "deleted_remote": 0,
 "conflicts": 0, "errors": [], "summary": "к нам: +3 ~0 −0; туда: +1 ~0 −0"}
```

Изменить подключение

```
PATCH /api/integrations/<id>/
```

Любые из полей: `direction`, `color`, `enabled`, `password` (новый пароль приложения; для CalDAV). Ответ `200`: `{"calendar": {...}}`.

Отключить

```
DELETE /api/integrations/<id>/?remove_events=1
```

С `remove_events=1` из нашего календаря удаляются события, импортированные из этого календаря; без параметра они остаются. Ответ `200`: `{"deleted": 3}`.

Синхронизировать сейчас

```
POST /api/integrations/<id>/sync/     — один календарь → {"calendar": {...}, "sync": {...}}
POST /api/integrations/sync/          — все включённые → {"results": [{"calendar": {...},
"sync": {...}}, ...]}
```

Ошибки внешнего сервиса не дают `5xx`: они попадают в `sync.errors` и `calendar.last_error`.

Вход через Google (для браузера)

```
GET /integrations/google/connect/     — перенаправляет на страницу согласия Google
GET /integrations/google/callback/    — сюда Google возвращает пользователя
```

После успешного входа пользователь возвращается на главную страницу с `?integrations=google`, токены временно хранятся в сессии до вызова `POST /api/integrations/`. Неудача — `?integrations=google-denied` или `?integrations=google-error`.

Приглашения

```
POST /api/events/<id>/invite/
```

Нужно право **Изменение**. Каждому приглашённому сотруднику (с e-mail) и гостю из `guest_emails` уходит отдельное письмо с файлом `invite.ics` (`METHOD:REQUEST`). Тело `{"cancel": true}` — отправить отмену (`METHOD:CANCEL`), например перед удалением события.

Ответ `200`:

```
{"sent": ["anna@corp.ru", "petr@rambler.ru"], "failed": {}}
```

Если адресатов нет — `{"sent": [], "message": "Некому отправлять: ..."}`. Участникам из адресной книги, уже получившим событие автоматически, повторно не пишется. `failed` — `{адрес: причина}` для писем, которые не ушли; если не ушло ни одно — `502` с причиной (например, «Почтовый сервер не настроен…»).

Адресная книга (AD / LDAP)

Подробно — в DIRECTORY.md. Поиск выполняется **на сервере каталога** при каждом запросе; контакты в базе календаря не хранятся. Доступно **любому вошедшему пользователю, независимо от роли**. Настройка каталогов — только администратору.

Поиск

```
GET /api/contacts/?q=текст&limit=20
```

`q` — от 2 символов (короче — пустой результат без обращения к серверу каталога). `limit` — до 100. Без `q` — только сведения о каталогах.

Поиск — по ФИО в любом порядке, e-mail, должности, подразделению, кабинету; если запрос похож на телефон — по цифрам номера в любом формате. Без учёта регистра, «ё» = «е». Одинаковый запрос 60 секунд отдаётся из кэша.

Ответ:

```
{
  "configured": true, "can_manage": false, "min_query": 2,
  "sources": [{"id": 1, "name": "YG", "kind": "ldap", "kind_label": "LDAP (...)", "enabled":
true}],
  "query": "петров", "total": 2,
  "results": [{
    "id": "1:6f1c…", "name": "Петров Иван Сергеевич", "last_name": "Петров", "first_name":
"Иван", "middle_name": "Сергеевич",
    "email": "i.petrov@domain.ru", "phone": "+7 (34922) 4-12-34", "mobile": "",
    "title": "Начальник отдела", "department": "Отдел ИТ", "company": "", "office": "305",
    "source": "YG", "user_id": 4, "exact": true
  }],
  "errors": []
}
```

- `id` — «:<идентификатор записи в каталоге>» (objectGUID / entryUUID / DN).
- `user_id` — пользователь календаря с тем же e-mail (или `null`).
- `exact` — полное совпадение фамилии, имени, ФИО или e-mail.
- `errors` — каталоги, которые не ответили: `[{"source": "YG", "message": "Каталог временно недоступен."}]`. Результаты остальных каталогов при этом возвращаются. Администратору (`can_manage: true`) в `message` — точная причина, а в `sources` — `host`, `port`, `base_dn`, `bind_dn`, `last_error`, `last_error_at`, `last_ok_at`. Пароль не отдаётся никогда.
- `configured: false` — ни один каталог не включён.

Настройка (только администратор)

Создание и изменение каталогов — в админке (там же кнопки «Проверить подключение» и «Проверить поиск»). Через API:

```
GET  /api/directory/sources/              — каталоги и их состояние
POST /api/directory/sources/<id>/test/    — проверить подключение
```

Ответ проверки:

```
{"ok": true, "message": "Подключение успешно. Найдено записей (первые): 5.", "found": 5,
 "samples": [{"dn": "uid=ipetrov,cn=domain.ru", "name": "Петров Иван", "email":
"i.petrov@domain.ru"}],
 "attributes": ["cn", "displayName", "givenName", "mail", "sn", "telephoneNumber"]}
```

При ошибке — `{"ok": false, "message": "...", "network": false, "details": []}` (код `200`). Если сервер недоступен по сети (`"network": true`), в `details` — диагностика: адрес в DNS, какие LDAP-порты отвечают и что поменять.

Не администратору оба адреса отвечают `403`.

Сводная таблица

Метод	Адрес	Нужное право	Успех
`GET`	`/api/events/`	Чтение (фильтрует список)	`200`
`POST`	`/api/events/`	Запись в календарь владельца	`201`
`GET`	`/api/events/report/?start=&end=`	Чтение (в отчет попадают доступные события, кроме выключенных в фильтре календарей)	`200`, файл `.docx`
`GET`	`/api/events/<id>/`	Чтение	`200`
`PUT`	`/api/events/<id>/`	Изменение (смена владельца — администратор)	`200`
`PATCH`	`/api/events/<id>/`	Изменение	`200`
`DELETE`	`/api/events/<id>/`	Изменение	`200`
`POST`	`/api/events/<id>/attachments/`	Изменение (или автор нового события с правом Запись)	`201`
`GET`	`/api/attachments/<id>/download/`	Чтение	`200`, файл
`DELETE`	`/api/attachments/<id>/`	Изменение	`200`
`GET`	`/api/reminders/`	любой вошедший пользователь (свои напоминания)	`200`
`GET`, `PATCH`	`/api/me/settings/`	любой вошедший пользователь (свои настройки)	`200`
`GET`	`/api/delegations/`	владелец календаря / администратор	`200`
`POST`	`/api/delegations/`	владелец календаря / администратор	`200`
`GET`	`/api/delegations/candidates/`	владелец календаря / администратор	`200`
`GET`	`/api/integrations/`	владелец своего календаря	`200`
`POST`	`/api/integrations/discover/`	владелец своего календаря	`200`
`POST`	`/api/integrations/`	владелец своего календаря	`201`
`PATCH`	`/api/integrations/<id>/`	свой подключённый календарь	`200`

Метод	Адрес	Нужное право	Успех
`DELETE`	`/api/integrations/<id>/`	свой подключённый календарь	`200`
`POST`	`/api/integrations/<id>/sync/`	свой подключённый календарь	`200`
`POST`	`/api/integrations/sync/`	—	`200`
`POST`	`/api/events/<id>/invite/`	Изменение	`200`
`GET`	`/api/contacts/?q=`	любой вошедший пользователь (поиск на сервере каталога)	`200`
`GET`	`/api/directory/sources/`	администратор	`200`
`POST`	`/api/directory/sources/<id>/test/`	администратор	`200`